Table of Contents

Table of Contents **1**
CaLLP Local Development Guide — MySQL **2**
1. Prerequisites 2
2. Repository layout 2
3. First-time setup 3
    3.1 Initialization 3
    3.2 Local database connection 5
    3.3 Verify initialization 6
4. Common commands 8
    4.1 Everyday Commands 8
    4.2 Reset Local Database (Destructive) 9
5. How initialization works 9
6. Troubleshooting 9
7. Repository and Security Rules 10

CaLLP Local Development Guide — MySQL

This guide sets up a **local MySQL 8.4.9 database** for CaLLP development using **Docker Compose**. Each developer runs an independent database on their own computer. *Nothing in this setup connects to or updates AWS RDS*.

1. Prerequisites

- Git and a local clone of the CaLLP repository.
- Docker Desktop (macOS/Windows) or Docker Engine with Docker Compose (Linux), installed and running.
- A terminal opened at the repository root (the directory containing `compose.yaml`).

Windows developers: Use Docker Desktop with the WSL 2 backend enabled. PowerShell is the recommended terminal for following this guide. Docker Compose commands are identical across supported operating systems unless otherwise indicated.

Verify Docker:

```shell
docker --version
docker compose version
docker info
```

2. Repository layout

```textproto
callp-app/
├── .gitattributes
├── .gitignore
├── .env.example
├── .env                    # Local only; never commit
├── compose.yaml
└── docker/
    └── mysql/
        ├── 00_initialize.sh
        └── init/
```

```
        ├── 01_schema_setup.sql
        ├── 02_triggers.sql
        ├── 03_seed_reserved_accounts.sql
        ├── 04_views.sql
        └── 05_populate.sql
```

- The five SQL files are part of the approved local initialization dataset. `05_populate.sql` contains `SOURCE` directives; the wrapper script handles its dependencies explicitly to avoid executing the schema reset twice.

3. First-time setup

3.1 Initialization

1. Open a terminal in the repository root.

2. If you do not already have a local `.env`, copy the template:

- **Do not overwrite an existing `.env`**; it may contain other local configuration, such as an API key. Instead, add the missing MySQL variables to it.

macOS / Linux / Windows PowerShell:

```shell
cp .env.example .env
```

Windows Command Prompt:

```shell
copy .env.example .env
```

3. Set two different, developer-specific passwords in `.env`:

```
Textproto
MYSQL_PASSWORD=replace_with_a_unique_local_password
MYSQL_ROOT_PASSWORD=replace_with_a_different_local_root_password
```

Generate two strong, unique passwords using a password manager or one of the following commands:

- Run the appropriate command twice to generate separate passwords for `MYSQL_PASSWORD` and `MYSQL_ROOT_PASSWORD`.
- Never reuse AWS RDS credentials. Keep `.env` private and excluded from Git.

macOS / Linux:

```
Shell
openssl rand -hex 24
```

- On macOS/Linux, restrict file permissions using `chmod 600 .env`.

Windows PowerShell:

```
Shell
$bytes = New-Object byte[] 24
$rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
try {
    $rng.GetBytes($bytes)
} finally {
    $rng.Dispose()
}
[BitConverter]::ToString($bytes).Replace('-', '').ToLowerInvariant()
```

4. Validate the Compose configuration without printing resolved secrets:

```
Shell
docker compose config --quiet
```

5. Start the database:

```
Shell
docker compose up -d
docker compose ps
```

- Wait until the MySQL container reports **`healthy`**. On a new data volume, initialization runs automatically.

3.2 Local database connection

Setting	Value
Engine	`MySQL 8.4.9`
Host (from your computer)	`127.0.0.1`
Port	`3306`
Database	`callp_local_db`
Application username	`callp_dev`
Application password	`MYSQL_PASSWORD` in your `.env`
Root password	`MYSQL_ROOT_PASSWORD` in your `.env`

6. Connect from the repository root:

```
Shell
docker compose exec mysql mysql -u callp_dev -p callp_local_db
```

- Enter the local application password when prompted. At the `mysql>` prompt, run SQL statements; to return to your normal terminal, enter `exit;`.
- The container port is published to **`127.0.0.1:3306` only**, not to the local network. The local MySQL instance is independent of the deployed AWS RDS database.

3.3 Verify initialization

7. Check initialization logs

- After starting MySQL for the first time, verify that all initialization scripts completed successfully:

macOS / Linux:

```shell
docker compose logs mysql | grep -E 'Initializing:|ERROR|completed'
```

Windows PowerShell:

```shell
docker compose logs mysql | Select-String 'Initializing:|ERROR|completed'
```

Expected output:

```shell
Initializing: 01_schema_setup.sql
Initializing: 02_triggers.sql
Initializing: 03_seed_reserved_accounts.sql
Initializing: 05_populate.sql
Initializing: 04_views.sql
CaLLP local MySQL initialization completed.
```

If any `ERROR` messages appear, inspect the full logs:

```shell
docker compose logs mysql
```

- A successful first-time setup includes the log messages `CaLLP local MySQL initialization completed.` and `MySQL init process done. Ready for start up.`
- The log may also report that the `/docker-entrypoint-initdb.d/sql` directory was ignored; that is expected because `00_initialize.sh` executes the SQL files itself.

8. After connecting with the MySQL client, verify:

```sql
SELECT VERSION();
SHOW FULL TABLES;
SELECT category_type, COUNT(*) FROM category GROUP BY category_type;
SELECT COUNT(*) FROM category_parent;
SELECT COUNT(*) FROM career_resource;
SELECT
    TRIGGER_NAME,
    CHARACTER_SET_CLIENT,
    COLLATION_CONNECTION
FROM information_schema.TRIGGERS
WHERE TRIGGER_SCHEMA = 'callp_local_db';
```

Expected results for the currently approved initialization scripts:

Check	Expected
MySQL version	8.4.9
Base tables	12
Views	2
Triggers	2
Degree programs	122
Skills	142
Career Fields	78
Category-parent relationships	581
Career Resources	11

- The triggers should use `utf8mb4` as their client character set and `utf8mb4_0900_ai_ci` as their connection collation.

4. Common commands

Run these from the `callp-app` repository root

4.1 Everyday Commands

Create/start MySQL in the background:

```shell
docker compose up -d
```

Show status and health:

```shell
docker compose ps
```

View recent MySQL logs:

```shell
docker compose logs --tail=100 mysql
```

Stop MySQL; retain container and data:

```shell
docker compose stop
```

Restart an existing stopped container:

```shell
docker compose start
```

Remove container/network; preserve named database volume:

```shell
docker compose down
```

4.2 Reset Local Database (Destructive)

```shell
docker compose down -v
docker compose up -d
```

- **Warning:** `down -v` deletes the named local MySQL data volume, including **all local test records and changes**. The next startup recreates the database using the SQL files and current `.env` passwords. Do not use it unless you intend to discard local data. This does not touch AWS RDS.
- Changing `MYSQL_PASSWORD` or `MYSQL_ROOT_PASSWORD` in `.env` does **NOT** update accounts already stored in an initialized database volume. Change passwords within MySQL or intentionally reset the local volume.

5. How initialization works

On the **first start with an empty data volume**, the official MySQL container creates `callp_local_db` and the `callp_dev` user. Then `00_initialize.sh` executes:

1. `01_schema_setup.sql` — creates tables and constraints (destructive if rerun against populated data).
2. `02_triggers.sql` — creates category validation triggers.
3. `03_seed_reserved_accounts.sql` — creates reserved system accounts.
4. `05_populate.sql` — loads the approved sample/reference dataset, with its nested `SOURCE` directives skipped because dependencies are already run.
5. `04_views.sql` — creates the two curator-visibility views.

The wrapper uses `utf8mb4` for its MySQL client sessions and a temporary restricted client configuration file for root authentication. Initialization **does not rerun** when restarting an existing database volume. Editing SQL initialization files will not automatically change an existing local database.

6. Troubleshooting

- **`docker: command not found`:** Ensure Docker is installed, then restart your terminal or check your shell `PATH`.
- **Docker daemon unavailable:** Start Docker Desktop and wait for the engine to become ready.
- **Port 3306 already in use:** Stop the conflicting local MySQL service or adjust the host port in `compose.yaml` and the local connection settings together.

- **`Access denied` for `callp_dev`:** Check that you entered `MYSQL_PASSWORD`, not the root password. Remember that changing `.env` does not rotate credentials on an existing volume.
- **Missing tables or incomplete data:** Inspect `docker compose logs mysql`. A healthy MySQL server does not by itself prove that initialization scripts succeeded. If initialization failed on a new volume, correct the cause before performing an intentional reset.
- **SQL file changes have no effect:** Scripts only run on first initialization of an empty volume. Do not reset a database containing work you need to retain.

7. Repository and Security Rules

- Commit `compose.yaml`, `.env.example`, `docker/mysql/00_initialize.sh`, and reviewed SQL initialization files.
- **Never commit** `.env`, credentials, private keys, local database exports, or unreviewed personal data.
- Keep the development SQL dataset anonymized and approved for repository distribution.
- Use different local credentials for each developer; do not reuse AWS credentials.
- Do not run these destructive initialization scripts against AWS RDS.
- Schema updates to RDS and Django integration are **out of scope** for this local Docker setup.
- Shell initialization scripts must use `LF` line endings for compatibility with Linux-based Docker containers. The repository enforces this through `.gitattributes`, ensuring consistent behavior across Windows, macOS, and Linux.